Academic Year 2025/2026

Module:J2EE, Java EE (JEE) ou Jakarta EE

Compte Rendu :

TP N° 3 –Login/Signup avec Sessions et Filtres

Réalisé par:
Taouni Ouiam

Supervisé par:
Pr. Mohamed CHERRADI

Table des matières

1. **Objectif du laboratoire**
2. **Architecture générale du projet**
3. **Flux utilisateur complet ET Implémentation des servlets et JSP**
   - 3.1 Page d'inscription (`signup.jsp`)
   - 3.2 Page de connexion (`login.jsp`)
   - 3.3 Page de bienvenue (`welcome.jsp`)
   - 3.4 Déconnexion (`LogoutServlet`)
   - 3.5 Filtre de sécurité (`AuthFilter`)
4. **Conclusion**

Objectif du laboratoire

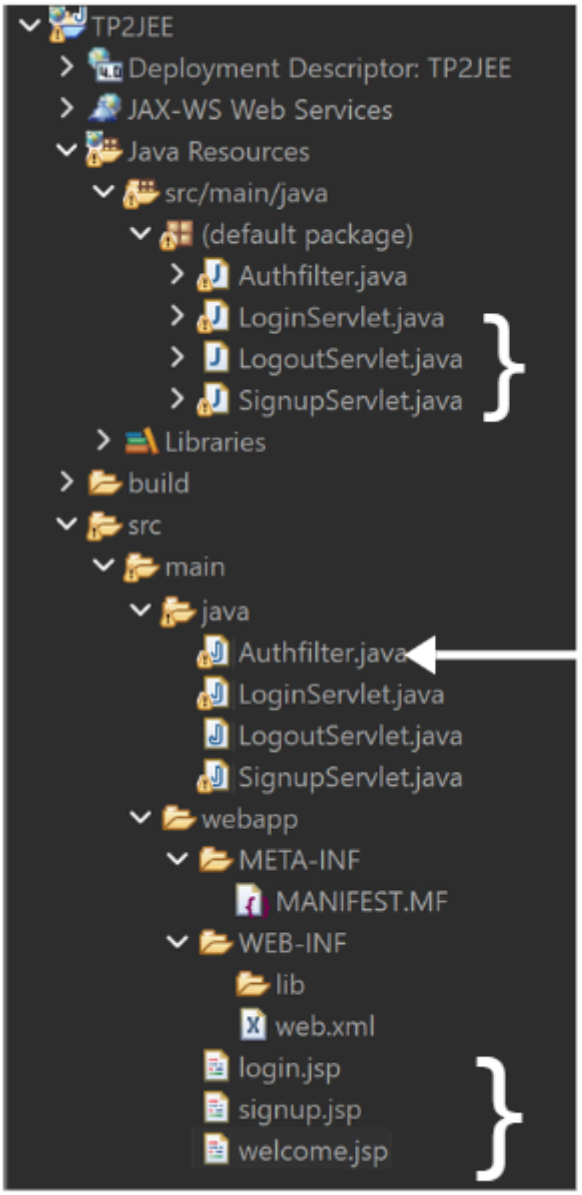
L’objectif de ce laboratoire était de créer un système d’authentification simple, tout en s’assurant que les pages sensibles soient protégées et ne puissent pas être consultées sans authentification.
Ce laboratoire nous a permis d’apprendre à implémenter les sessions pour gérer l’état des utilisateurs et à utiliser des filtres pour sécuriser l’accès aux pages.

1-Matériel et technologies utilisés

- **Java EE** (Servlets, JSP)
- **ServletContext** pour le stockage partagé
- **HttpSession** pour la gestion des sessions
- **JSP** pour les pages web (login.jsp, signup.jsp, welcome.jsp)
- **Filter (AuthFilter)** pour protéger les pages accessibles uniquement après authentification
- IDE : Eclipse / IntelliJ IDEA
- Serveur : Tomcat

2-Architecture du projet

Ce projet est un **Dynamic Web Project Java EE**, combinant JSP, servlets et filtres pour créer un système d'authentification sécurisé.



Les servlets (SignupServlet, LoginServlet, LogoutServlet) gèrent la logique côté serveur : traitement des formulaires, vérification des informations d'utilisateur et gestion des sessions.

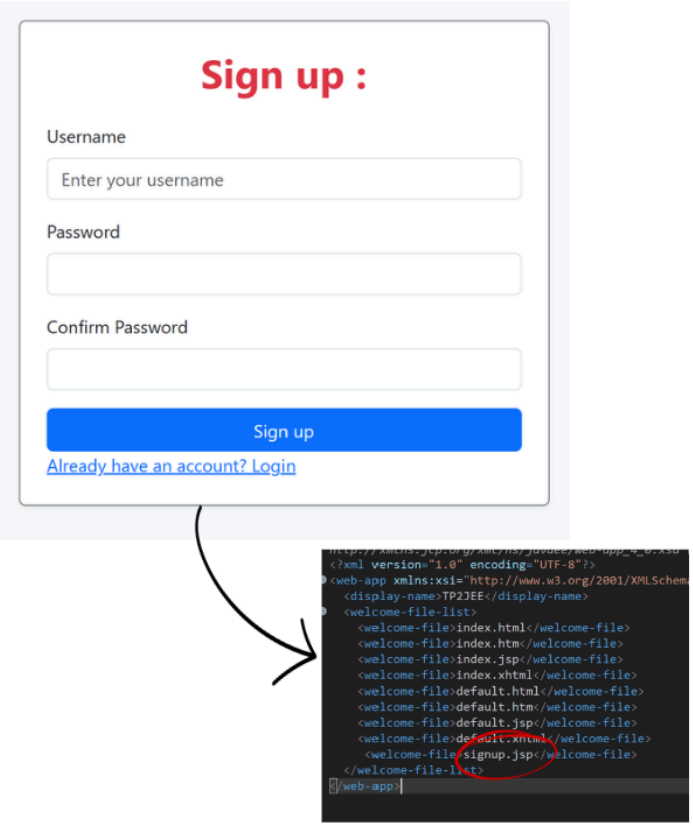
protège les pages sensibles pour empêcher l'accès direct via URL

Les JSP servent d'interface utilisateur pour la connexion, l'inscription et la page de bienvenue

3-Flux utilisateur et Implémentation :

1-Page d’accueil – priorité à l’inscription :

Lorsque l’utilisateur ouvre l’application pour la première fois, il est automatiquement redirigé vers signup.jsp, configuré dans le web.xml comme page d’accueil par défaut.



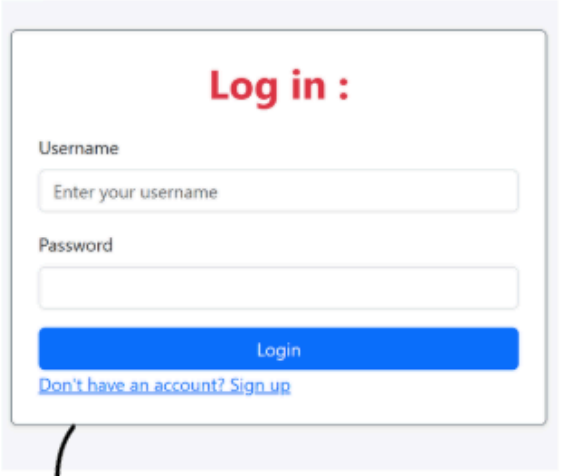
2-Inscription (signup.jsp → SignupServlet) :

- L'utilisateur saisit un nom d'utilisateur et un mot de passe.
- Le SignupServlet vérifie si l'utilisateur existe déjà dans la Map partagée (ServletContext).
- Si l'utilisateur n'existe pas, il est ajouté à la Map et une **session est créée** (session.setAttribute("username", ...)).
- L'utilisateur est alors redirigé vers welcome.jsp.
- Sous le formulaire d'inscription, un bouton "Déjà un compte ? Se connecter" permet aux utilisateurs existants d'accéder à login.jsp.

```java
protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

    doGet(request, response);

    String username = request.getParameter("username");
    String password = request.getParameter("password");

    ServletContext context = getServletContext();
    Map<String,String> users = (Map<String,String>) context.getAttribute("users");
    if(users == null){
        users = new HashMap<>();
        context.setAttribute("users", users);
    }

    if (users.containsKey(username)) {
        response.sendRedirect("signup.jsp?error=exists");
    }

    users.put(username, password);

    HttpSession session = request.getSession();
    session.setAttribute("user", username);

    response.sendRedirect("welcome.jsp");
}
```

3-Connexion (login.jsp → LoginServlet) :

- L'utilisateur saisit son nom d'utilisateur et son mot de passe.
- Le LoginServlet vérifie les informations dans la Map partagée.
- Si les informations sont correctes, une **session est créée ou reprise** et l'utilisateur est redirigé vers welcome.jsp.
- Si les informations sont incorrectes, l'utilisateur est renvoyé vers login.jsp avec un message d'erreur.




```java
protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
    // TODO Auto-generated method stub
    doGet(request, response);

    String username = request.getParameter("username");
    String password = request.getParameter("password");

    Map<String , String> users = (Map<String , String>) getServletContext().getAttribute("users");

    if(users != null && users.containsKey(username) && users.get(username).equals(password)) {
        HttpSession session = request.getSession();
        session.setAttribute("user", username);

        response.sendRedirect("welcome.jsp");
    }else {
        response.sendRedirect("signup.jsp?error=invalid");
    }

}
```

4-Inscription (signup.jsp → SignupServlet) :

Toutes les requêtes vers welcome.jsp passent par ce filtre.

Il vérifie si une **session existe** et contient l'attribut "user".

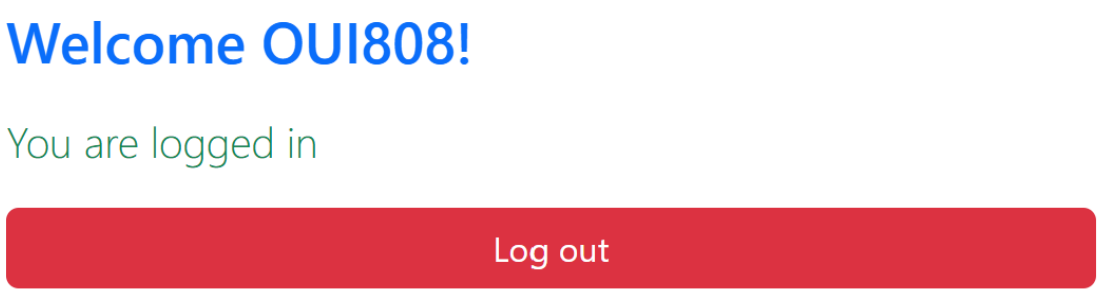
Si la session est valide → la requête est autorisée (chain.doFilter(req, res)).

Sinon → l'utilisateur est redirigé vers login.jsp.

Cela empêche l'accès direct à la page de bienvenue via l'URL, même si l'utilisateur tape l'adresse manuellement après s'être déconnecté.

```
public void doFilter(ServletRequest req, ServletResponse res, FilterChain chain)
        throws IOException, ServletException {

    HttpServletRequest request = (HttpServletRequest) req;
    HttpServletResponse response = (HttpServletResponse) res;

    HttpSession session = request.getSession(false);


    if (session != null && session.getAttribute("user") != null) {

        chain.doFilter(req, res);
    } else {
        response.sendRedirect(request.getContextPath() + "/login.jsp");
    }
}
```

5-welcome page et logout



Le bouton **"Log out"** présent dans welcome.jsp envoie simplement une requête POST vers le **LogoutServlet**, qui fait une seule action :

- **Invalider la session** (session.invalidate()) pour déconnecter l'utilisateur.
- **Rediriger l'utilisateur** vers login.jsp ou signup.jsp selon le flux choisi.

Ainsi, la déconnexion est gérée **entièrement par le servlet**, et après avoir cliqué sur le bouton, l'utilisateur **ne peut plus accéder aux pages protégées** sans se reconnecter, grâce au filtre AuthFilter.

```java
protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
    // TODO Auto-generated method stub
    response.getWriter().append("Served at: ").append(request.getContextPath());
    HttpSession session = request.getSession(false);

    if (session != null) {
        session.invalidate();
    }

    response.sendRedirect("login.jsp");
}
```

Conclusion

Ce laboratoire m’a permis de comprendre comment créer et gérer des sessions en Java EE, le rôle des servlets pour gérer la logique côté serveur, ainsi que l’utilisation des filtres pour sécuriser les pages sensibles. J’ai également appris à implémenter un système simple de login et signup sans base de données, en utilisant une Map partagée dans le ServletContext. Ce travail m’a montré qu’il est essentiel, pour de bonnes pratiques, de toujours protéger les pages critiques avec un filtre et de vérifier l’existence des sessions avant d’afficher le contenu aux utilisateurs.